18.2.1. OpenSHMEM

intro_shmem - Introduction to the OpenSHMEM programming model

18.2.1.1. DESCRIPTION

The SHMEM programming model consists of library routines that provide low-latency, high-bandwidth communication for use in highly parallelized scalable programs. The routines in the OpenSHMEM application programming interface (API) provide a programming model for exchanging data between cooperating parallel processes. The resulting programs are similar in style to Message Passing Interface (MPI) programs. The SHMEM API can be used either alone or in combination with MPI routines in the same parallel program.

An OpenSHMEM program is SPMD (single program, multiple data) in style. The SHMEM processes, called processing elements or PEs, all start at the same time and they all run the same program. Usually the PEs perform computation on their own subdomains of the larger problem and periodically communicate with other PEs to exchange information on which the next computation phase depends.

The OpenSHMEM routines minimize the overhead associated with data transfer requests, maximize bandwidth and minimize data latency. Data latency is the period of time that starts when a PE initiates a transfer of data and ends when a PE can use the data. OpenSHMEM routines support remote data transfer through put operations, which transfer data to a different PE, get operations, which transfer data from a different PE, and remote pointers, which allow direct references to data objects owned by another PE. Other operations supported are collective broadcast and reduction, barrier synchronization, and atomic memory operations. An atomic memory operation is an atomic read-and-update operation, such as a fetch-and-increment, on a remote or local data object.

18.2.1.2. OPENSHMEM ROUTINES

This section lists the significant OpenSHMEM message-passing routines.

PE queries

main

- C/C++ only:
    - *_num_pes*(3)
    - *_my_pe*(3)
- Fortran only:
    - *NUM_PES*(3)
    - *MY_PE*(3)

Elemental data put routines

- C/C++ only:
    - shmem_double_p(3)
    - shmem_float_p(3)
    - shmem_int_p(3)
    - shmem_long_p(3)
    - shmem_short_p.*(3)

Block data put routines

- C/C++ and Fortran:
    - shmem_put32(3)
    - shmem_put64(3)
    - shmem_put128(3)
- C/C++ only:
    - shmem_double_put(3)
    - shmem_float_put(3)
    - shmem_int_put(3)
    - shmem_long_put(3)
    - shmem_short_put.*(3)
- Fortran only:
    - shmem_complex_put(3)
    - shmem_integer_put(3)
    - shmem_logical_put(3)
    - shmem_real_put(3)

Elemental data get routines

main

- C/C++ only:
    - shmem_double_g(3)
    - shmem_float_g(3)
    - shmem_int_g(3)
    - shmem_long_g(3)
    - shmem_short_g(3)

Block data get routines

- C/C++ and Fortran:
    - shmem_get32(3)
    - shmem_get64(3)
    - shmem_get128(3)
- C/C++ only:
    - shmem_double_get(3)
    - shmem_float_get(3)
    - shmem_int_get(3)
    - shmem_long_get(3)
    - shmem_short_get(3)
- Fortran only:
    - shmem_complex_get(3)
    - shmem_integer_get(3)
    - shmem_logical_get(3)
    - shmem_real_get(3)

Strided put routines

- C/C++ and Fortran:
    - shmem_iput32(3)
    - shmem_iput64(3)
    - shmem_iput128(3)
- C/C++ only:
    - shmem_double_iput(3)
    - shmem_float_iput(3)
    - shmem_int_iput(3)
    - shmem_long_iput(3)
    - shmem_short_iput(3)
- Fortran only:
    - shmem_complex_iput(3)
    - shmem_integer_iput(3)
    - shmem_logical_iput(3)
    - shmem_real_iput(3)

Strided get routines

- C/C++ and Fortran:
    - shmem_iget32(3)
    - shmem_iget64(3)
    - shmem_iget128(3)
- C/C++ only:
    - shmem_double_iget(3)
    - shmem_float_iget(3)
    - shmem_int_iget(3)
    - shmem_long_iget(3)
    - shmem_short_iget(3)
- Fortran only:
    - shmem_complex_iget(3)
    - shmem_integer_iget(3)
    - shmem_logical_iget(3)
    - shmem_real_iget(3)

Point-to-point synchronization routines

main

- C/C++ only:
    - shmem_int_wait(3)
    - shmem_int_wait_until(3)
    - shmem_long_wait(3)
    - shmem_long_wait_until(3)
    - shmem_longlong_wait(3)
    - shmem_longlong_wait_until(3)
    - shmem_short_wait(3)
    - shmem_short_wait_until(3)
- Fortran:
    - shmem_int4_wait(3)
    - shmem_int4_wait_until(3)
    - shmem_int8_wait(3)
    - shmem_int8_wait_until(3)

Barrier synchronization routines

- C/C++ and Fortran:
    - shmem_barrier_all(3)
    - shmem_barrier(3)

Atomic memory fetch-and-operate (fetch-op) routines

- C/C++ and Fortran:
    - shmem_swap

Reduction routines

- C/C++ only:
    - shmem_int_and_to_all(3)
    - shmem_long_and_to_all(3)
    - shmem_longlong_and_to_all(3)
    - shmem_short_and_to_all(3)
    - shmem_double_max_to_all(3)
    - shmem_float_max_to_all(3)
    - shmem_int_max_to_all(3)
    - shmem_long_max_to_all(3)
    - shmem_longlong_max_to_all(3)
    - shmem_short_max_to_all(3)
    - shmem_double_min_to_all(3)
    - shmem_float_min_to_all(3)
    - shmem_int_min_to_all(3)
    - shmem_long_min_to_all(3)
    - shmem_longlong_min_to_all(3)
    - shmem_short_min_to_all(3)
    - shmem_double_sum_to_all(3)
    - shmem_float_sum_to_all(3)
    - shmem_int_sum_to_all(3)
    - shmem_long_sum_to_all(3)
    - shmem_longlong_sum_to_all(3)
    - shmem_short_sum_to_all(3)
    - shmem_double_prod_to_all(3)
    - shmem_float_prod_to_all(3)
    - shmem_int_prod_to_all(3)
    - shmem_long_prod_to_all(3)
    - shmem_longlong_prod_to_all(3)
    - shmem_short_prod_to_all(3)
    - shmem_int_or_to_all(3)
    - shmem_long_or_to_all(3)
    - shmem_longlong_or_to_all(3)
    - shmem_short_or_to_all(3)
    - shmem_int_xor_to_all(3)
    - shmem_long_xor_to_all(3)
    - shmem_longlong_xor_to_all(3)
    - shmem_short_xor_to_all(3)
- Fortran only:

    - shmem_int4_and_to_all(3)
    - shmem_int8_and_to_all(3)
    - shmem_real4_max_to_all(3)
    - shmem_real8_max_to_all(3)
    - shmem_int4_max_to_all(3)
    - shmem_int8_max_to_all(3)
    - shmem_real4_min_to_all(3)
    - shmem_real8_min_to_all(3)
    - shmem_int4_min_to_all(3)
    - shmem_int8_min_to_all(3)
    - shmem_real4_sum_to_all(3)
    - shmem_real8_sum_to_all(3)
    - shmem_int4_sum_to_all(3)
    - shmem_int8_sum_to_all(3)
    - shmem_real4_prod_to_all(3)
    - shmem_real8_prod_to_all(3)
    - shmem_int4_prod_to_all(3)
    - shmem_int8_prod_to_all(3)
    - shmem_int4_or_to_all(3)
    - shmem_int8_or_to_all(3)
    - shmem_int4_xor_to_all(3)
    - shmem_int8_xor_to_all(3)

Broadcast routines

- C/C++ and Fortran:
    - shmem_broadcast32(3)
    - shmem_broadcast64(3)

Cache management routines

- C/C++ and Fortran:
    - shmem_udcflush(3)
    - shmem_udcflush_line(3)

Byte-granularity block put routines

- C/C++ and Fortran
    - shmem_putmem(3)
    - shmem_getmem(3)
- Fortran only:
    - shmem_character_put(3)
    - shmem_character_get(3)

Collect routines

- C/C++ and Fortran:
    - shmem_collect32(3)
    - shmem_collect64(3)
    - shmem_fcollect32(3)
    - shmem_fcollect64(3)

Atomic memory fetch-and-operate (fetch-op) routines

- C/C++ only:
    - shmem_double_swap(3)
    - shmem_float_swap(3)
    - shmem_int_cswap(3)
    - shmem_int_fadd(3)
    - shmem_int_finc(3)
    - shmem_int_swap(3)
    - shmem_long_cswap(3)
    - shmem_long_fadd(3)
    - shmem_long_finc(3)
    - shmem_long_swap(3)
    - shmem_longlong_cswap(3)
    - shmem_longlong_fadd(3)
    - shmem_longlong_finc(3)
    - shmem_longlong_swap(3)
- Fortran only:
    - shmem_int4_cswap(3)
    - shmem_int4_fadd(3)
    - shmem_int4_finc(3)
    - shmem_int4_swap(3)
    - shmem_int8_swap(3)
    - shmem_real4_swap(3)
    - shmem_real8_swap(3)
    - shmem_int8_cswap(3)

Atomic memory operation routines

- Fortran only:
    - shmem_int4_add(3)
    - shmem_int4_inc(3)

Remote memory pointer function

- C/C++ and Fortran:
    - shmem_ptr(3)

Reduction routines

main

- C/C++ only:
    - shmem_longdouble_max_to_all(3)
    - shmem_longdouble_min_to_all(3)
    - shmem_longdouble_prod_to_all(3)
    - shmem_longdouble_sum_to_all(3)
- Fortran only:
    - shmem_real16_max_to_all(3)
    - shmem_real16_min_to_all(3)
    - shmem_real16_prod_to_all(3)
    - shmem_real16_sum_to_all(3)

Accessibility query routines

- C/C++ and Fortran:
    - shmem_pe_accessible(3)
    - shmem_addr_accessible(3)

Symmetric Data Objects

Consistent with the SPMD nature of the OpenSHMEM programming model is the concept of symmetric data objects. These are arrays or variables that exist with the same size, type, and relative address on all PEs. Another term for symmetric data objects is “remotely accessible data objects”. In the interface definitions for OpenSHMEM data transfer routines, one or more of the parameters are typically required to be symmetric or remotely accessible.

The following kinds of data objects are symmetric:

- Fortran data objects in common blocks or with the SAVE attribute. These data objects must not be defined in a dynamic shared object (DSO).
- Non-stack C and C++ variables. These data objects must not be defined in a DSO.
- Fortran arrays allocated with *shpalloc*(3)
- C and C++ data allocated by *shmalloc*(3)

Collective Routines

Some SHMEM routines, for example, shmem_broadcast(3) and shmem_float_sum_to_all(3), are classified as collective routines because they distribute work across a set of PEs. They must be called concurrently by all PEs in the active set defined by the PE_sta
PE_size argument triplet. The following man pages describe the OpenSHMEI
routines:

main

- shmem_and(3)
- shmem_barrier(3)
- shmem_broadcast(3)
- shmem_collect(3)
- shmem_max(3)
- shmem_min(3)
- shmem_or(3)
- shmem_prod(3)
- shmem_sum(3)
- shmem_xor(3)

18.2.1.3. USING THE SYMMETRIC WORK ARRAY, PSYNC

Multiple pSync arrays are often needed if a particular PE calls as OpenSHMEM collective routine twice without intervening barrier synchronization. Problems would occur if some PEs in the active set for call 2 arrive at call 2 before processing of call 1 is complete by all PEs in the call 1 active set. You can use shmem_barrier(3) or shmem_barrier_all(3) to perform a barrier synchronization between consecutive calls to OpenSHMEM collective routines.

There are two special cases:

- The shmem_barrier(3) routine allows the same pSync array to be used on consecutive calls as long as the active PE set does not change.
- If the same collective routine is called multiple times with the same active set, the calls may alternate between two pSync arrays. The SHMEM routines guarantee that a first call is completely finished by all PEs by the time processing of a third call begins on any PE.

Because the SHMEM routines restore pSync to its original contents, multiple calls that use the same pSync array do not require that pSync be reinitialized after the first call.

18.2.1.4. SHMEM ENVIRONMENT VARIABLES

This section lists the significant SHMEM environment variables.

- **SMA_VERSION** print the library version at start-up.
- **SMA_INFO** print helpful text about all these environment variables.
- **SMA_SYMMETRIC_SIZE** number of bytes to allocate for the symmetric heap.
- **SMA_DEBUG** enable debugging messages.

main

The first call to SHMEM must be *start_pes*(3). This routines initialize the SHMEM runtime.

Calling any other SHMEM routines beforehand has undefined behavior. Multiple calls to this routine is not allowed.

18.2.1.5. COMPILING AND RUNNING OPENSHMEM PROGRAMS

The OpenSHMEM specification is silent regarding how OpenSHMEM programs are compiled, linked and run. This section shows some examples of how wrapper programs could be utilized to compile and launch applications. The commands are styled after wrapper programs found in many MPI implementations.

The following sample command line demonstrates running an OpenSHMEM Program using a wrapper script (**oshrun** in this case):

- C/C++:

```
oshcc c_program.c
```

- FORTRAN:

```
oshfort fortran_program.f
```

The following sample command line demonstrates running an OpenSHMEM Program assuming that the library provides a wrapper script for such purpose (named **oshrun** for this example):

```
oshrun -n 32 ./a.out
```

18.2.1.6. EXAMPLES

Example 1: The following Fortran OpenSHMEM program directs all PEs to sum simultaneously the numbers in the VALUES variable across all PEs:

```
PROGRAM REDUCTION
  REAL VALUES, SUM
  COMMON /C/ VALUES
  REAL WORK

  CALL START_PES(0)
  VALUES = MY_PE()
  CALL SHMEM_BARRIER_ALL ! Synchronize all PEs
  SUM = 0.0
  DO I = 0, NUM_PES()-1
    CALL SHMEM_REAL_GET(WORK, VALUES, 1, I) ! Get next value
    SUM = SUM + WORK                ! Sum it
  ENDDO
  PRINT *, 'PE ', MY_PE(), ' COMPUTED SUM=', SUM
  CALL SHMEM_BARRIER_ALL
END
```

Example 2: The following C OpenSHMEM program transfers an array of 10 longs from PE 0 to PE 1:

```
#include <mpp/shmem.h>

main() {
  long source[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
  static long target[10];

  shmem_init();
  if (shmem_my_pe() == 0) {
    /* put 10 elements into target on PE 1 */
    shmem_long_put(target, source, 10, 1);
  }
  shmem_barrier_all(); /* sync sender and receiver */
  if (shmem_my_pe() == 1)
    printf("target[0] on PE %d is %d\n", shmem_my_pe(), target[0]);
}
```

See also

The following man pages also contain information on OpenSHMEM routines. See the specific man pages for implementation information.

shmem_add(3) shmem_and(3) shmem_barrier(3) shmem_barrier_all(3) shmem_broadcast(3) shmem_cache(3) shmem_collect(3) shmem_cswap(3) shmem_fadd(3) shmem_fence(3) shmem_finc(3) shmem_get(3) shmem_iget(3) shmem_inc(3) shmem_iput(3) shmem_lock(3) shmem_max(3) shmem_min(3) shmem_my_pe(3) shmem_or(3) shmem_prod(3)

main

shmem_quiet(3) shmem_short_g(3) shmem_short_p(3) shmem_sum(3) shmem_swap(3) shmem_wait(3) shmem_xor(3) shmem_pe_accessible(3) shmem_addr_accessible(3) shmem_init(3) shmem_malloc(3) *shmem_my_pe*(3) *shmem_n_pes*(3)

main